

Radar

Secure Radar Over Web Implementation

Overview

- Setup Linux Based Computer(Pi 5)
- Setup the example software
 - Setup all required libraries(does not work with newest version)
- Modify Example Software to work better on host
- Implement Remote access points
 - Python Script to open the file system to the web
- Rewrite C++ code to have Web access

Setup the example

- GitHub repos with detailed directions

Setup Procedure

Installing Furuno Sensor SDK

How to Install:

1. Start with a Fresh Raspberry Pi OS Install

Run the following command to upgrade the system after clean install:

```
sudo apt upgrade
```

2. Extract NAVNETDLL_RADAR_ARMLINUX_64bit_01.95.zip

Inside the zip file, there is a tar.gz file. Extract it to a folder inside the root directory `/`. *Note that the exact version number and OS may be different.

3. Extract the File to `/usr/lib/NAVNETDLL`

Use these commands to extract the file:

```
cd /  
sudo tar -xzf /path_to_SDK_tar_file/NAVNETDLL_RADAR_ARMLINUX_64bit_01.95.tar.gz -C /
```

*Ensure that the `path_to_SDK_tar_file` is replaced with your actual path. The necessary files will be installed in `/usr/lib/NAVNETDLL/`.

4. IP Address Setting

The PC that installs Furuno Sensor SDK and connects to the sensor device must be set to an IP address of the device to be connected. Use a Class B address (172.31.xxx.xxx, subnet mask: 255.255.0.0). [172.31.1.100 in my example]

5. Firewall Settings

Furuno Sensor SDK uses ports 10000 to 10199 for both TCP and UDP communication with the sensor device.

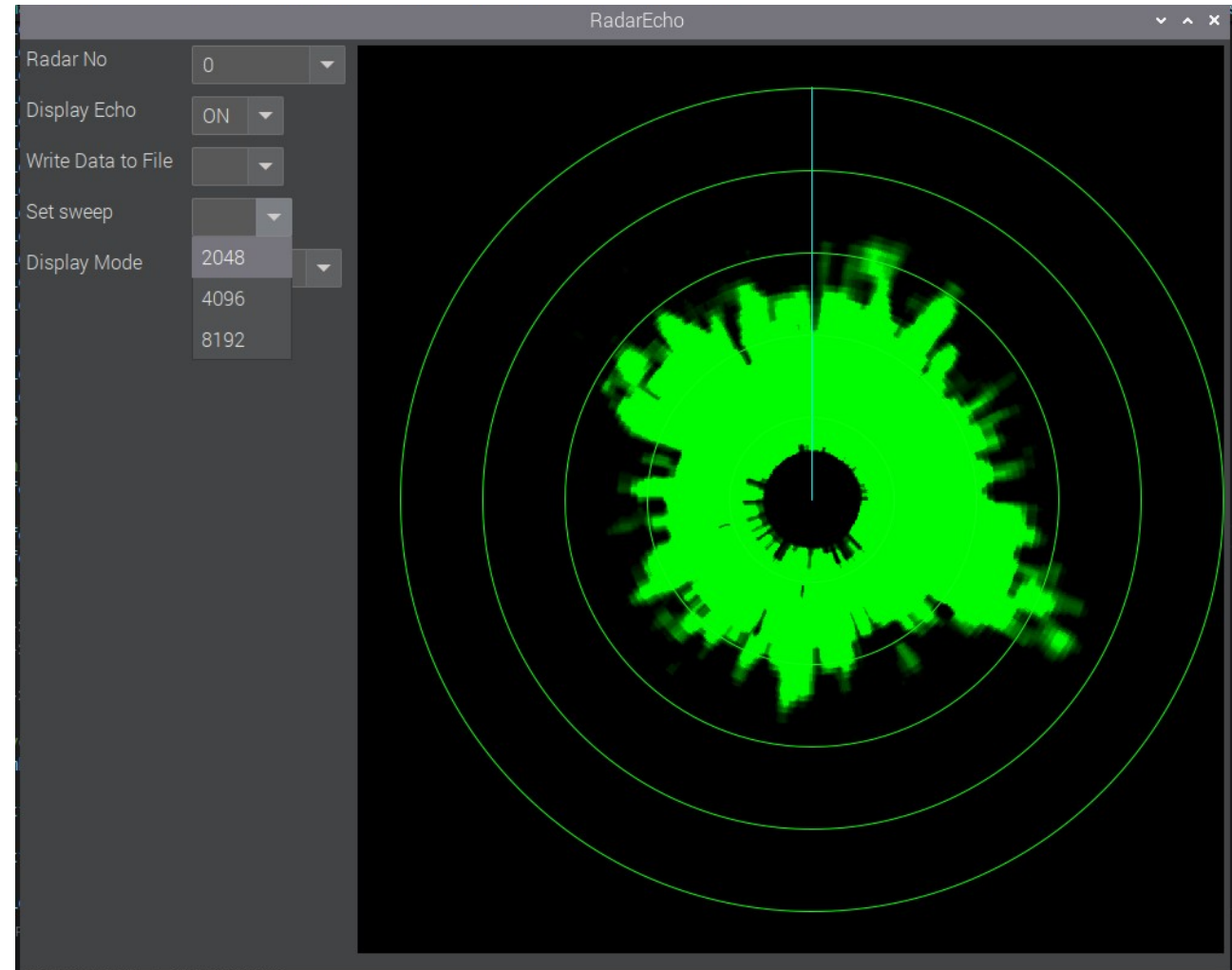
Compile Ulsample Program

The operation of this sample program was confirmed with the wxWidgets 3.1.5 GUI library.

Operation in other versions is not guaranteed AND WILL NOT WORK.

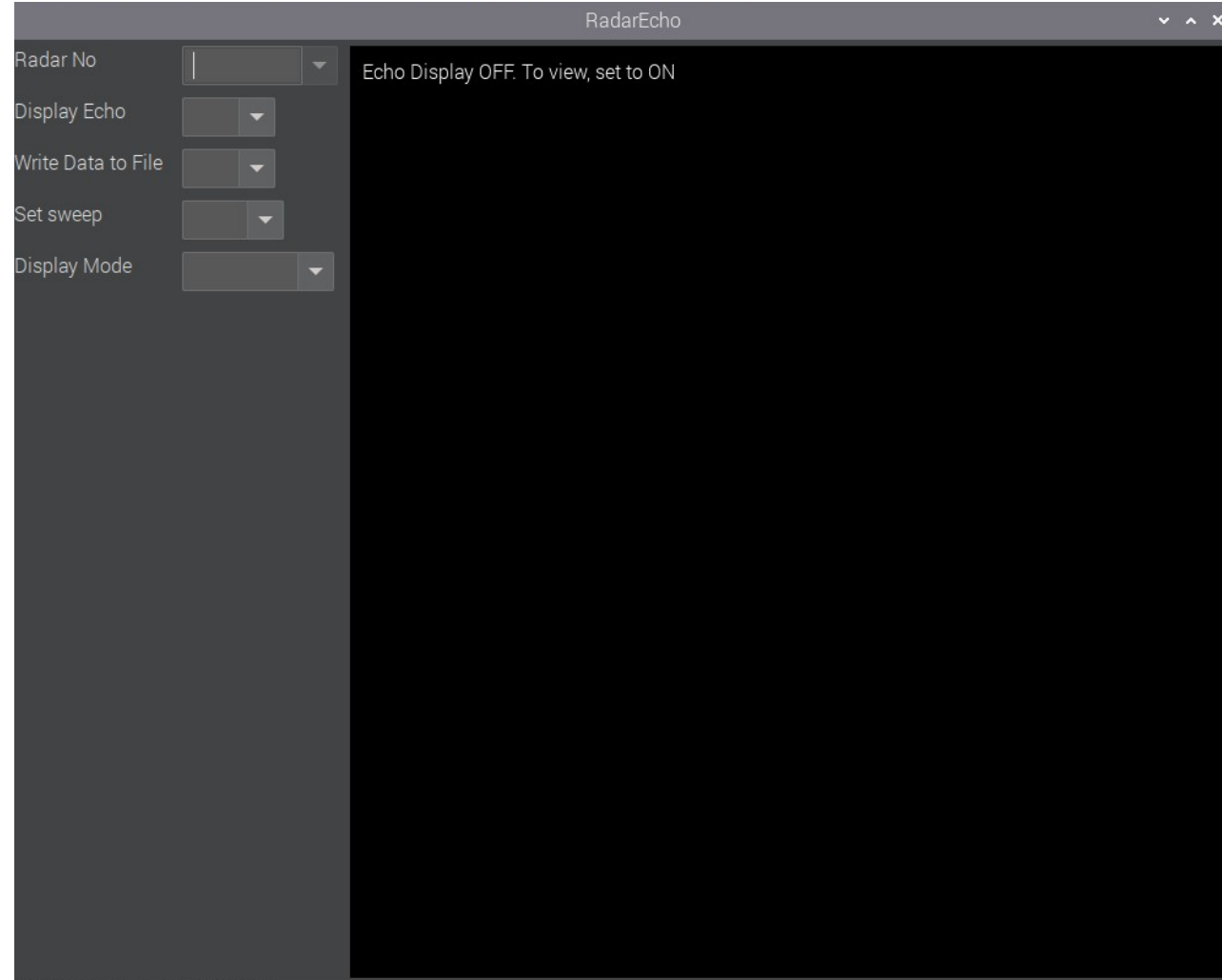
Adding Features to host machine

- Added File-writing capabilities
- Used Binary to limit size
- Looked into compression methods
 - 7z was better than any method I developed
- Created Python script to create a 'live' radar view
- Issue: only 1024 bytes are used instead of 2048.



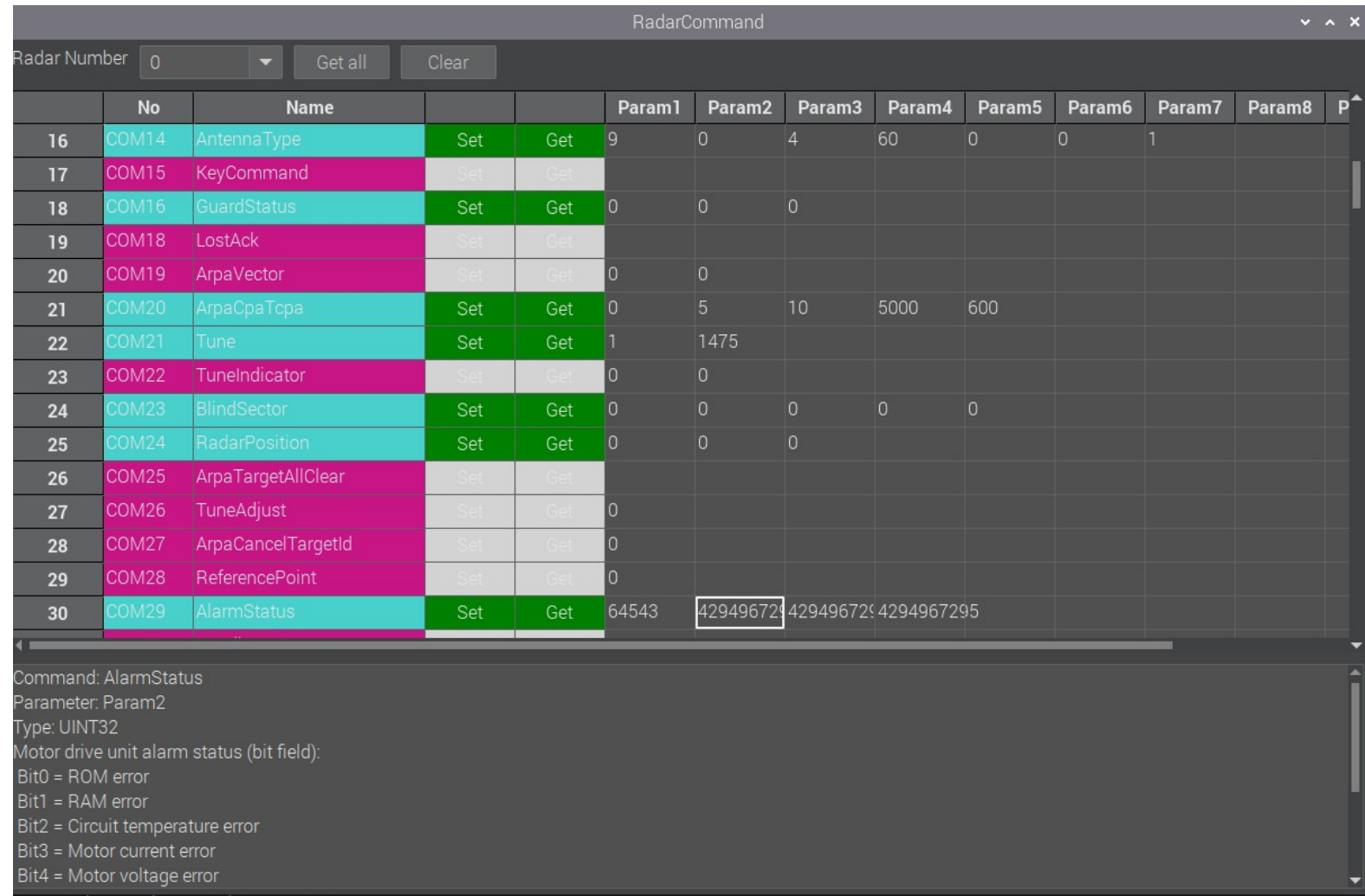
Adding Features

- Allows for screen to be turned off to make the system more efficient
 - Automatically load the correct radar, correct sweep, vendor and radar files upon starting the program to eliminate needless steps
- Starts the radar transmission mode on start-up as well.
- Shuts radar down when closing this window



Adding Features to host machine

- Wrote descriptions for EVERY single input into the radar
- Each param has a short description
 - Most commands have a description, but not complete.



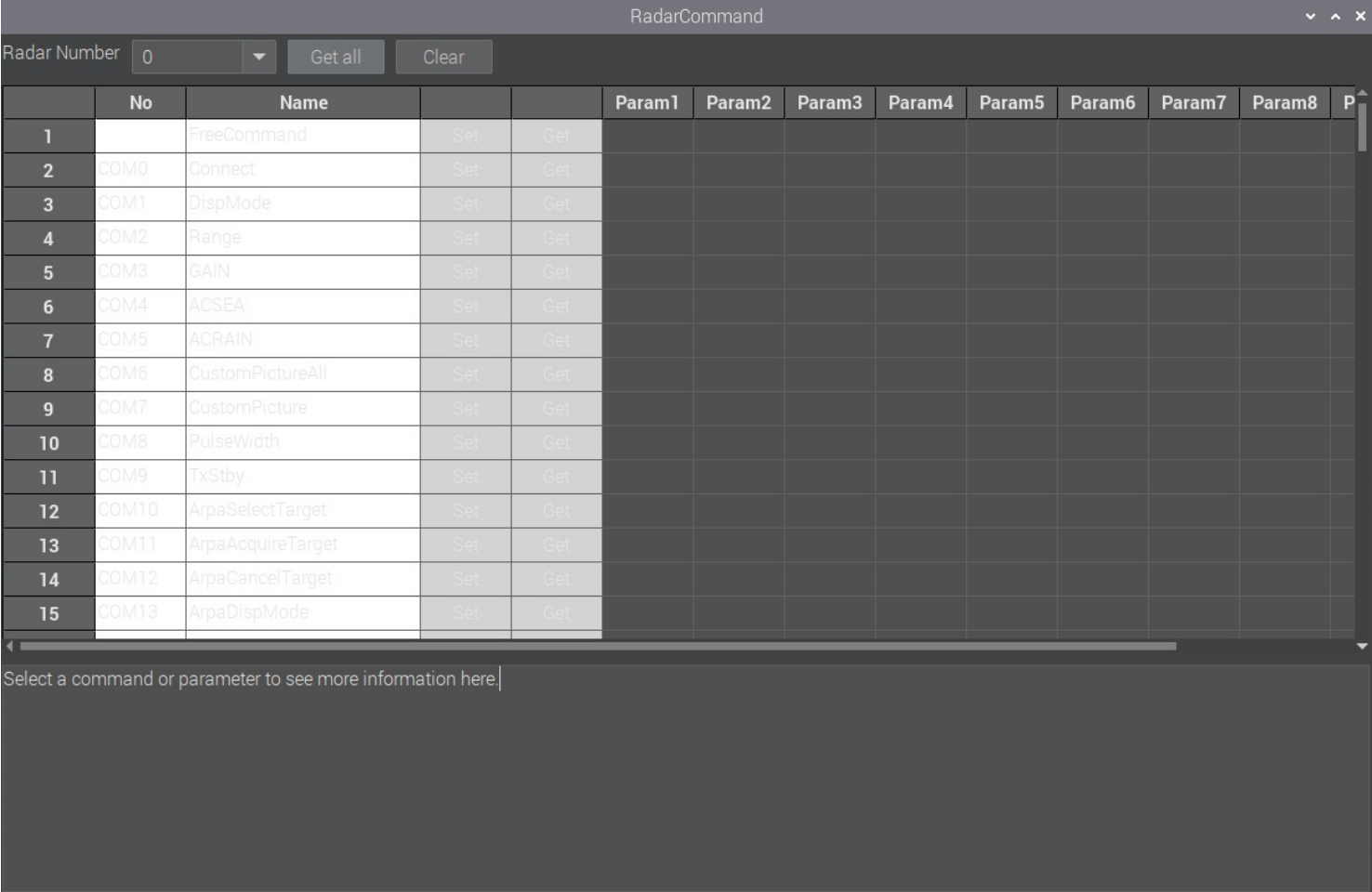
The screenshot shows a software interface titled "RadarCommand". At the top, there is a "Radar Number" dropdown menu set to "0", and two buttons labeled "Get all" and "Clear". Below this is a table with columns for "No", "Name", "Set", "Get", and eight "Param" columns (Param1 to Param8). The table lists 16 commands, numbered 16 to 30. Each command has a unique color (cyan or magenta) and a "Set" and "Get" status (green or grey). The "Param" columns contain numerical values. At the bottom of the interface, there is a text area displaying details for the selected command, "AlarmStatus".

	No	Name	Set	Get	Param1	Param2	Param3	Param4	Param5	Param6	Param7	Param8	P
16	COM14	AntennaType	Set	Get	9	0	4	60	0	0	1		
17	COM15	KeyCommand	Set	Get									
18	COM16	GuardStatus	Set	Get	0	0	0						
19	COM18	LostAck	Set	Get									
20	COM19	ArpaVector	Set	Get	0	0							
21	COM20	ArpaCpaTcpa	Set	Get	0	5	10	5000	600				
22	COM21	Tune	Set	Get	1	1475							
23	COM22	TuneIndicator	Set	Get	0	0							
24	COM23	BlindSector	Set	Get	0	0	0	0	0				
25	COM24	RadarPosition	Set	Get	0	0	0						
26	COM25	ArpaTargetAllClear	Set	Get									
27	COM26	TuneAdjust	Set	Get	0								
28	COM27	ArpaCancelTargetId	Set	Get	0								
29	COM28	ReferencePoint	Set	Get	0								
30	COM29	AlarmStatus	Set	Get	64543	42949672	42949672	42949672	95				

Command: AlarmStatus
Parameter: Param2
Type: UINT32
Motor drive unit alarm status (bit field):
Bit0 = ROM error
Bit1 = RAM error
Bit2 = Circuit temperature error
Bit3 = Motor current error
Bit4 = Motor voltage error

Adding Features

- Highlights the control panel to show that data has not been checked yet
- Highlights each cell to show that they are editable/not editable



The screenshot shows a software interface titled "RadarCommand". At the top, there is a "Radar Number" dropdown menu set to "0", and two buttons: "Get all" and "Clear". Below this is a table with 15 rows and 14 columns. The first two columns are "No" and "Name". The next two columns are "Set" and "Get". The remaining 10 columns are labeled "Param1" through "Param10". The table contains 15 rows of data, each with a unique command name and its corresponding parameters. Below the table, there is a text input field with the placeholder text "Select a command or parameter to see more information here".

	No	Name	Set	Get	Param1	Param2	Param3	Param4	Param5	Param6	Param7	Param8	Param9	Param10
1		FreeCommand	Set	Get										
2	COM0	Connect	Set	Get										
3	COM1	DispMode	Set	Get										
4	COM2	Range	Set	Get										
5	COM3	GAIN	Set	Get										
6	COM4	ACSEA	Set	Get										
7	COM5	ACRAIN	Set	Get										
8	COM6	CustomPictureAll	Set	Get										
9	COM7	CustomPicture	Set	Get										
10	COM8	PulseWidth	Set	Get										
11	COM9	TxStby	Set	Get										
12	COM10	ArpaSelectTarget	Set	Get										
13	COM11	ArpaAcquireTarget	Set	Get										
14	COM12	ArpaCancelTarget	Set	Get										
15	COM13	ArpaDispMode	Set	Get										

Select a command or parameter to see more information here

Compression Methods I tried

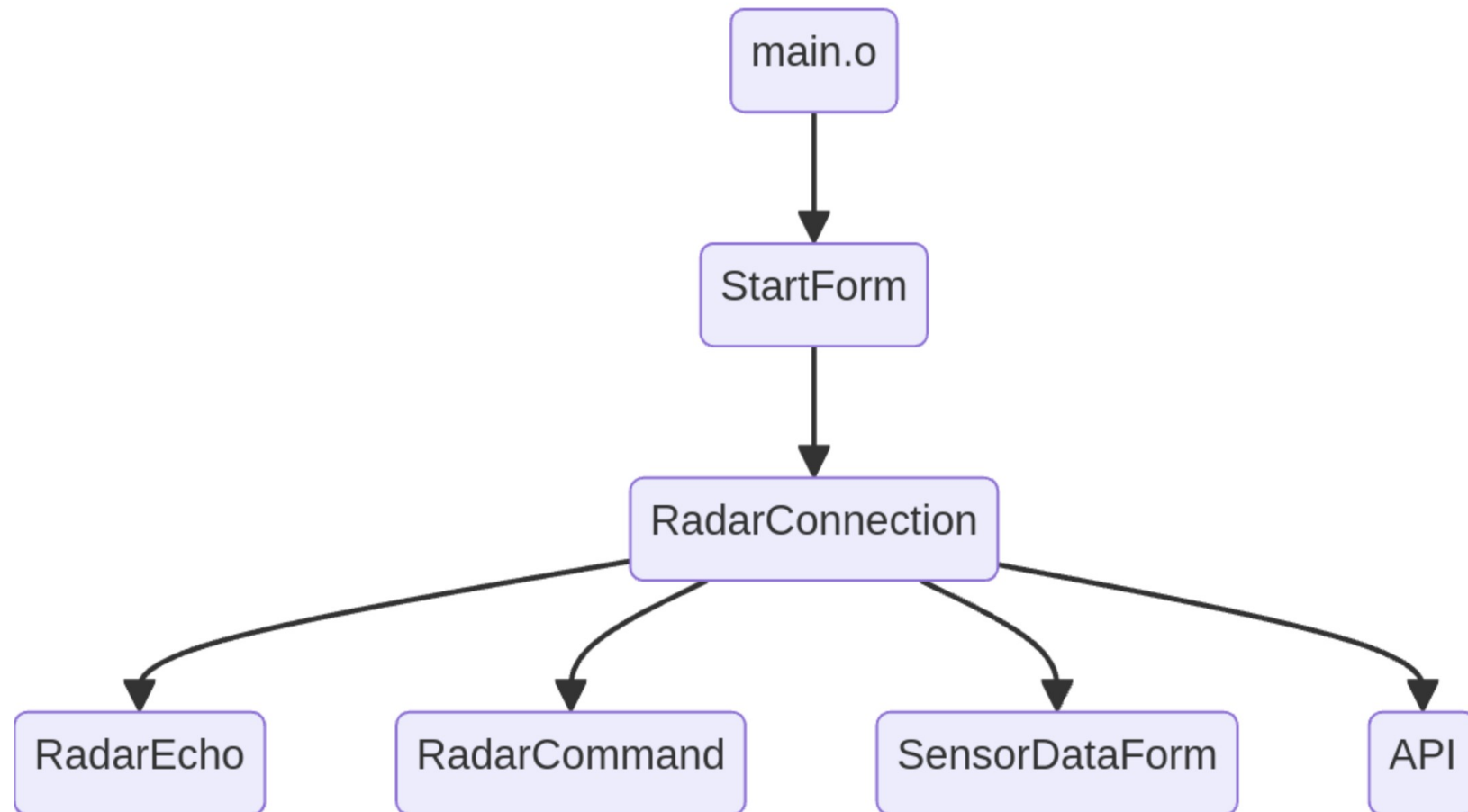
- 1. Run-Length Encoding (RLE):** A simple lossless compression algorithm that stores consecutive repeating elements (runs) as a single value and a count.
- 2. Arithmetic Encoding:** A lossless algorithm that represents a sequence of symbols as a fractional value within the interval $[0,1)$. It works well for inputs with known probability distributions.
- 3. Context-based Compression:** Uses the surrounding context to predict the next symbol or data value.
- 4. Huffman Encoding:** A lossless algorithm that assigns shorter codes to more frequent symbols and longer codes to less frequent symbols, resulting in efficient compression. It uses a binary tree structure to encode data.

Chosen Compression

- Most likely do not need lossless compression
- Will probably select a lossy method since screen is small anyway.
- Best compression was 7z with a very high compression rate.

Helper funcs Program Structure

- Binary to CSV
- Binary to PNG
- CSV to PNG
- Binary to video



Current/Next Steps

- How to send data over internet?
- How to integrate into existing software?
- Which features to keep?
- Assumed Bandwidth?
- Documentation. Ongoing
- How to implement Smart Gain Adjustments?